Question 2: Update and Delete Users Using Express, MongoDB and Mongoose

Problem Statement

Extend the Express.js and MongoDB application created in Question 1 to perform **update and delete operations** on existing user data.

The application must use **Mongoose models** to update and delete records from MongoDB.

Students must maintain the same proper folder structure and should not place schema, model, or router logic directly inside `server.js`.

Required Folder Structure

```
project/
│
├── server.js
│
├── schema/
│   └── userSchema.js
│
├── model/
│   └── userModel.js
│
└── router/
    └── userRouter.js
```

Requirements

1. MongoDB Connection

Connect the Express application to MongoDB using Mongoose.

Display:

MongoDB connected successfully

when the connection is successful.

2. Use Existing Schema and Model

Use:

schema/userSchema.js

for the schema definition and:

model/userModel.js

for the Mongoose model.

Do not recreate the schema or model inside the router or server file.

3. Update User

Create:

PATCH `/api/users/:id`

The endpoint should update an existing user's information using the MongoDB document ID.

Example:

PATCH /api/users/68abc123...

Request body:

```
{
  "age": 23,
  "course": "MCA"
}
```

The API should:

- Read the ID from `req.params`.
- Read updated information from `req.body`.
- Find the corresponding user.
- Update the user in MongoDB.
- Return an appropriate success response.

Example response:

```
{
  "message": "User updated successfully"
}
```

4. Delete User

Create:

DELETE `/api/users/:id`

The endpoint should delete the user whose ID is provided in the URL.

Example:

DELETE /api/users/68abc123...

The API should:

- Read the ID from `req.params`.
- Find the corresponding user.
- Delete the user from MongoDB.
- Return an appropriate success response.

Example:

```
{
  "message": "User deleted successfully"
}
```

5. Error Handling

Handle situations such as:

- Invalid MongoDB ID.
- User not found.
- Database error.
- Invalid request data.

Return meaningful HTTP status codes and messages.

Example API Flow

POST
  ↓
Create User
  ↓
MongoDB

GET
  ↓
Retrieve Users
  ↓

MongoDB

PATCH /api/users/:id
↓
Find User
↓
Update User
↓
MongoDB

DELETE /api/users/:id
↓
Find User
↓
Delete User
↓
MongoDB

Rubric – Question 2

Total: 10 Marks

Criteria	Description	Marks
Folder Structure	Properly maintains `schema`, `model`, `router`, and `server.js` structure	1
Schema & Model Usage	Correctly uses the separate Mongoose schema and model files	1
Router Setup	Correctly creates and connects the user router with Express	1
PATCH API	Correctly implements `PATCH /api/users/:id`	2
Update Database Operation	Correctly finds and updates the required MongoDB document using Mongoose	1
DELETE API	Correctly implements `DELETE /api/users/:id`	2
Delete Database Operation	Correctly removes the required document from MongoDB	1
Error Handling	Handles invalid ID, missing user, and database errors appropriately	1

Total	**10**